

NEXUS Gaming

Radar automático de novedades

Diseño e implementación de una automatización en React

Estudiante: _____

Curso / grupo: _____ Fecha: _____

1. Problema y objetivo

Una tienda de componentes gamer necesita destacar novedades sin que el visitante revise manualmente todos los productos o recargue la página. NEXUS Gaming automatiza la consulta de un catálogo local y la selección de hasta tres lanzamientos recientes con stock, ordenados por valoración. La tienda también permite buscar, filtrar, ordenar y agregar productos a un carrito de demostración.

Los productos, precios, valoraciones y fechas son ficticios. “Mejores” significa mejor valorados dentro de este catálogo. La aplicación no consulta lanzamientos reales en Internet ni realiza ventas.

2. Disparador y secuencia de tareas

El primer ciclo se programa al montar React. Después, un temporizador `setInterval` dispara una consulta cada 10 segundos. Cada ciclo ejecuta `fetch('/api/products')`; la API de Node lee `db.json`, devuelve los productos y React valida la respuesta. Un límite de 6 segundos cancela solicitudes que no terminan.

El algoritmo conserva productos con stock positivo cuya fecha de lanzamiento esté entre hoy y hace 90 días, ambos inclusive, calculados en UTC. Ordena por valoración descendente, luego por fecha más reciente y, ante otro empate, por ID ascendente. Finalmente toma los tres primeros y actualiza el ranking, catálogo, hora e historial. Una lista válida sin novedades produce éxito con un ranking vacío.

3. Estados de la automatización

Estado	Comportamiento visible
Inactivo	Antes del primer ciclo o al pausar; no se programan nuevas consultas durante la pausa.
En ejecución	Se consulta y valida la API; un candado evita peticiones simultáneas.
Éxito	Se actualizan los datos y se espera el siguiente disparador.
Error	Se muestra la causa, se conserva la última lectura válida y se reintenta automáticamente.

Del workflow al código

4. Hooks, componentes y limpieza

El proyecto usa Vite y React. El hook personalizado `useRadar` agrupa la automatización: `useState` conserva estado, datos, historial y configuración; `useEffect` programa el primer ciclo y el intervalo; `useRef` permite simular un único fallo sin añadir otro renderizado. Los botones de pausa y actualización modifican el estado que controla el efecto.

El retorno de `useEffect` ejecuta `clearInterval`, limpia los temporizadores y aborta la petición mediante `AbortController`. Una bandera `disposed` evita que respuestas tardías actualicen un efecto cancelado. El bloque `finally` libera el candado y limpia el `timeout`. El contador visual del panel tiene su propio intervalo de un segundo y su correspondiente limpieza.

La interfaz se divide en componentes: `ProductCard` se reutiliza para cada producto; `ProductArt` genera ilustraciones locales; `RadarPanel` presenta resultados y controles; `Cart` maneja el diálogo del carrito. Las reglas de validación y clasificación están separadas en `src/lib/catalog.js`. El servidor utiliza únicamente módulos de Node.

5. Manejo de errores y workflow

Se comprueba el estado HTTP y se rechazan datos incompletos, IDs duplicados, fechas imposibles y valores numéricos inválidos. Los errores de red, HTTP, JSON y tiempo de espera llevan al estado `Error`. La ruta `/api/demo-error` devuelve HTTP 503 para demostrar esta rama. El ciclo siguiente vuelve a consultar la ruta normal sin intervención del usuario.

El diagrama adjunto, `workflow.svg` y su versión Mermaid `workflow.md`, representa inicio, disparador, petición, decisión sobre la respuesta, clasificación, caminos de éxito/error, espera y repetición. También documenta pausa, reanudación y limpieza, en correspondencia con el efecto implementado.

6. Verificación y demostración

Las pruebas automatizadas cubren límites de 90 días, fechas futuras, stock agotado, desempates, catálogo vacío y validación. Una prueba de integración comprueba que la API relea cambios del archivo y se recupera después de JSON inválido. Se verificaron compilación y análisis estático, además de carga, pausa mayor a 10 segundos, reanudación, error y recuperación, búsqueda, filtros, carrito y ancho móvil en Chrome.

Para demostrar el proceso, se cambia una valoración o el stock en `db.json` y se espera el próximo ciclo. Luego se pausa, se reanuda y se simula un fallo. El historial y las capturas permiten observar el resultado. El archivo `demonstracion.md` contiene el guion reproducible.

7. Alcance y conclusión

La aplicación transforma una tarea repetitiva en un flujo de estados, eventos y operaciones asíncronas con limpieza. El catálogo requiere mantenimiento manual; la consulta y clasificación son automáticas mientras la página permanece abierta y activa. Las fechas de ejemplo deben actualizarse si la demostración se realiza más adelante. El carrito no persiste al recargar ni procesa pagos.